

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	YAVANIKA S	Reg. No.:	192521258
Section / Batch:	ITA0512	Date:	24.08.2026

Code of Conduct

I **Yavanika S [192521258]** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: YAVANIKA S

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Question 6 — Real-Time Object Detection Pipeline

A system must:

- A. Capture video frames
- B. Detect objects using a trained model
- C. Draw bounding boxes and labels
- D. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

1. Problem Understanding

Category	Details
Input	Live video stream from a camera device (frame-by-frame image data)
Input	A pre-trained object detection model (e.g., MobileNet-SSD / YOLO-Tiny)
Output	Same video stream displayed in real time, each frame annotated with bounding boxes + class label + confidence score for every detected object
Constraint 1	Efficiency — every frame must be captured, processed and displayed fast enough to look real time (target: no visible lag, ideally ≥ 15 -20 FPS)
Constraint 2	Multiplicity — a single frame may contain zero, one, or many objects; the algorithm must loop over an unknown, variable number of detections without breaking
Assumption	The detection model and its class labels are already trained and available to load; this task designs the run-time pipeline, not the model training

2. Modular System Design

The pipeline is broken into small, single-purpose modules so each one can be written, tested and optimized independently. MAIN() only orchestrates the loop — it does not do any processing itself.

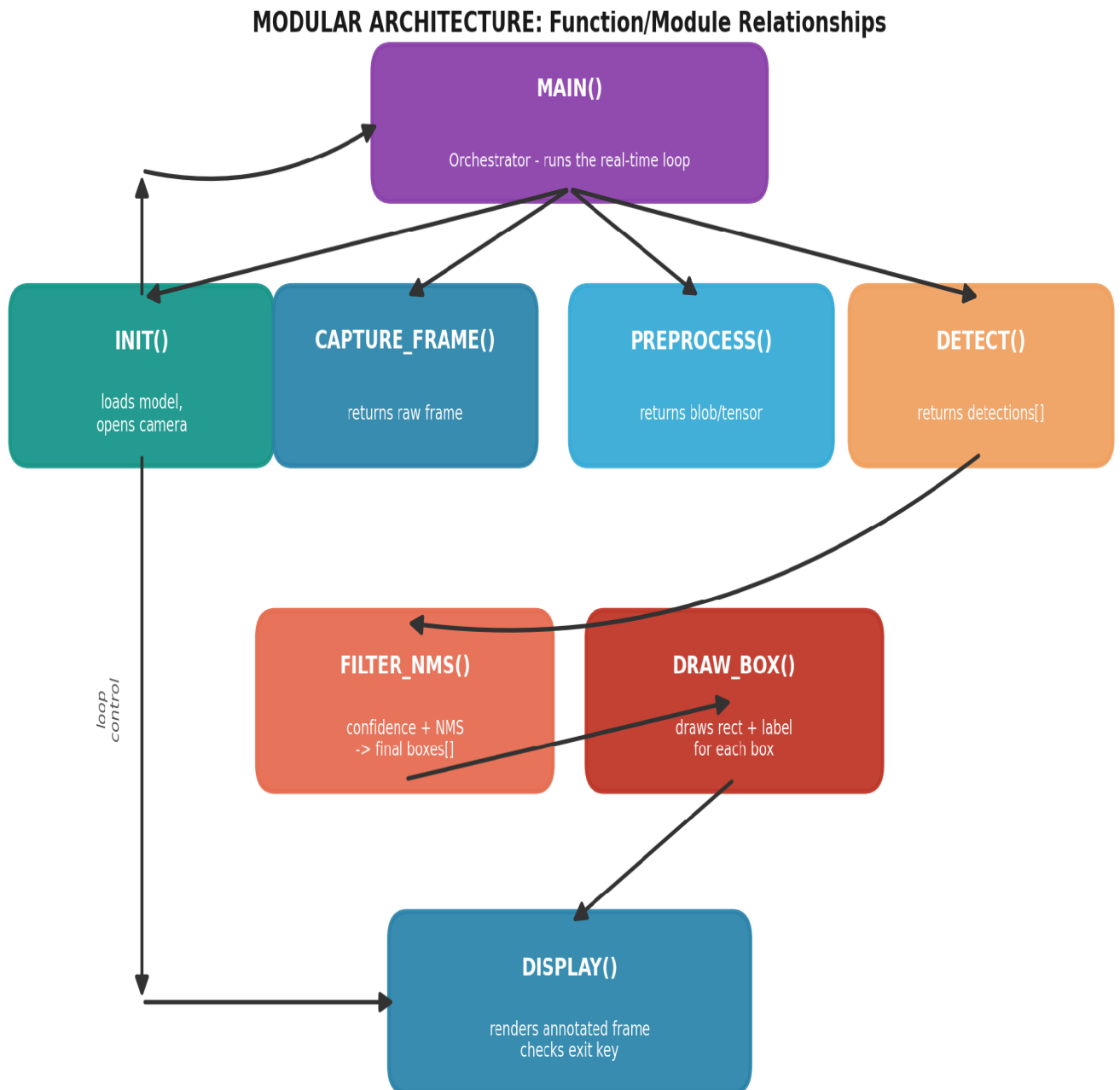


Figure 1: Modular architecture — function responsibilities and data flow

3. Modular Pseudocode

Pseudocode is organized into 7 modules, called in order by MAIN(). Comments (//) justify key decisions inline.

MODULE 1 — INIT()

FUNCTION INIT():

```
model <- LOAD_MODEL("trained_detector.weights") // load once, not per frame
classList <- LOAD_LABELS("classes.txt")
camera <- OPEN_CAMERA(source = 0)
IF camera NOT opened THEN
    PRINT "Error: camera unavailable"
    RETURN FAILURE
RETURN model, classList, camera
```

MODULE 2 — CAPTURE_FRAME(camera)

FUNCTION CAPTURE_FRAME(camera):

```
success, frame <- camera.READ()
IF NOT success THEN
    RETURN NULL // triggers loop exit in MAIN
RETURN frame
```

MODULE 3 — PREPROCESS(frame)

FUNCTION PREPROCESS(frame):

```
resized <- RESIZE(frame, width = 300, height = 300) // fixed input size -> constant
inference cost
blob <- CONVERT_TO_BLOB(resized, scale = 1/255, meanSubtract = TRUE)
RETURN blob
```

MODULE 4 — DETECT(model, blob)

FUNCTION DETECT(model, blob):

```
model.SET_INPUT(blob)
rawDetections <- model.FORWARD() // single forward pass per frame
RETURN rawDetections // array of {classId, confidence, box}
```

MODULE 5 — FILTER_NMS(rawDetections, threshold)

FUNCTION FILTER_NMS(rawDetections, confThreshold = 0.5, nmsThreshold = 0.4):

```
kept <- EMPTY LIST
FOR EACH d IN rawDetections: // handles ANY number of objects, 0..N
    IF d.confidence >= confThreshold THEN
        APPEND d TO kept
finalBoxes <- NON_MAX_SUPPRESSION(kept, nmsThreshold) // removes
duplicate/overlapping boxes
RETURN finalBoxes
```

MODULE 6 — DRAW_BOX(frame, finalBoxes, classList)

FUNCTION DRAW_BOX(frame, finalBoxes, classList):

```
FOR EACH box IN finalBoxes:           // loop scales to multiple objects automatically
    label <- classList[box.classId] + " " + ROUND(box.confidence * 100) + "%"
    DRAW_RECTANGLE(frame, box.x, box.y, box.w, box.h, color =
    COLOR_FOR(box.classId))
    DRAW_TEXT(frame, label, position = (box.x, box.y - 5))

RETURN frame
```

MODULE 7 — DISPLAY(frame)

FUNCTION DISPLAY(frame):

```
SHOW_WINDOW("Live Detection", frame)
key <- WAIT_KEY(1)           // 1 ms wait -> keeps loop non-blocking / real time

RETURN key
```

MAIN() — efficient real-time loop

FUNCTION MAIN():

```
model, classList, camera <- INIT()
frameCount <- 0

WHILE camera IS opened:
    frame <- CAPTURE_FRAME(camera)
    IF frame IS NULL THEN
        BREAK           // camera disconnected / end of stream
    frameCount <- frameCount + 1
    IF frameCount MOD SKIP_RATE == 0 THEN // frame skipping -> efficiency
        (see Section 6)
        blob <- PREPROCESS(frame)
        rawDetections <- DETECT(model, blob)
        finalBoxes <- FILTER_NMS(rawDetections)
        frame <- DRAW_BOX(frame, finalBoxes, classList)
    key <- DISPLAY(frame)
    IF key == 'q' THEN
        BREAK

RELEASE(camera)
CLOSE_ALL_WINDOWS()
```

4. Algorithm Flowchart

The flowchart below traces exactly the control flow written in MAIN() above — every decision diamond corresponds to an IF/WHILE in the pseudocode.

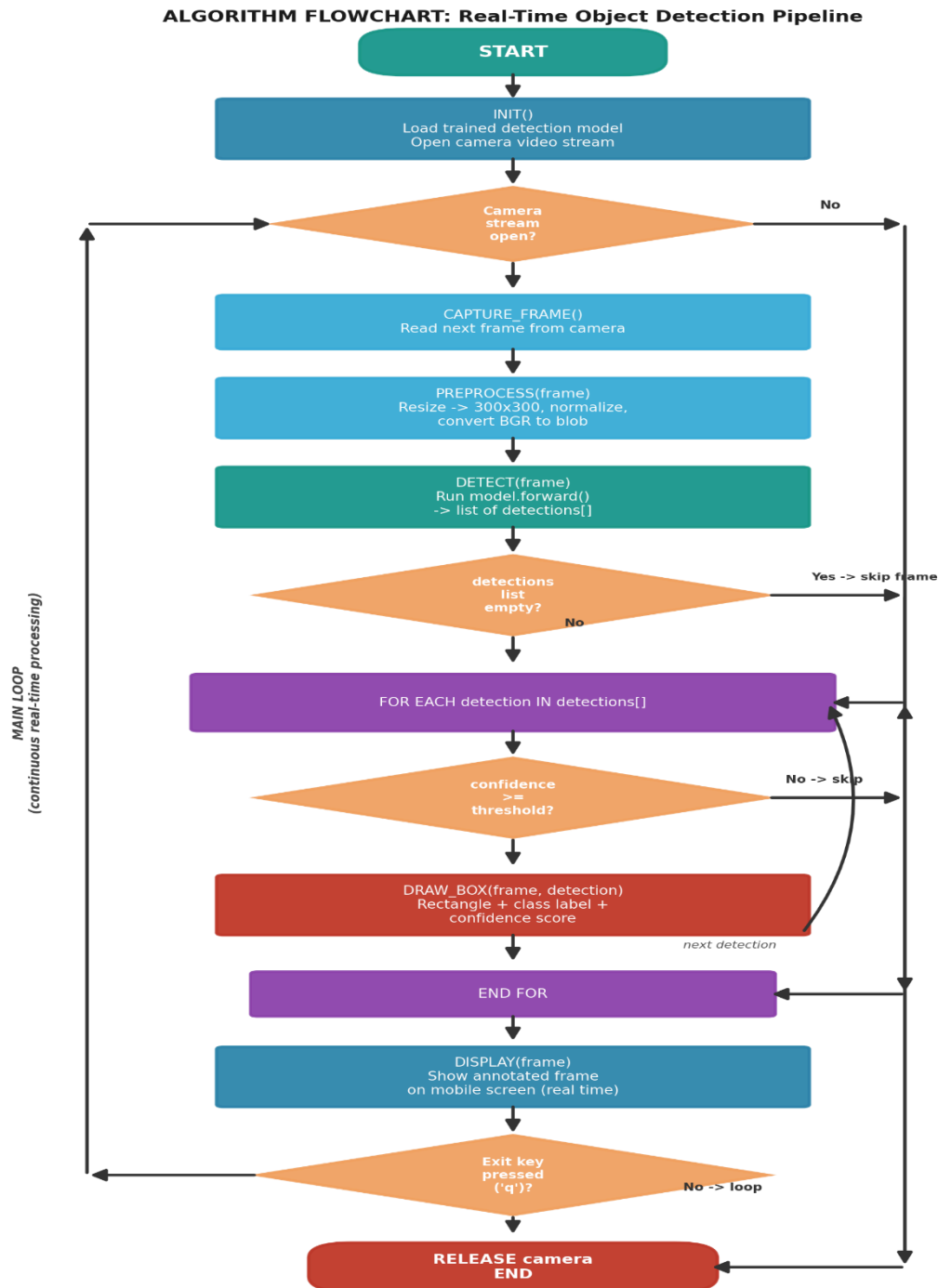


Figure 2: Control flow of the real-time detection loop

5. Use of Control Structures

- WHILE loop — in MAIN(); runs the pipeline continuously frame after frame until the camera closes or the user quits (what makes it "real time")
- IF / conditional — checks a missing frame, frameCount MOD SKIP_RATE, confidence threshold, and the exit key; controls efficiency and clean exits
- FOR EACH loop — in FILTER_NMS() and DRAW_BOX(); iterates over detections[] whether it holds 0, 1 or many entries, satisfying "handle multiple objects"
- Functions / modules — INIT, CAPTURE_FRAME, PREPROCESS, DETECT, FILTER_NMS, DRAW_BOX, DISPLAY; each stage is isolated and testable, so MAIN() stays short
- BREAK — in MAIN(); exits the WHILE loop immediately on camera failure or user quit, instead of a flag variable

6. Practical Techniques for Efficiency & Multiple Objects

- Fixed small input size (300x300 blob) in PREPROCESS() — keeps inference cost constant regardless of camera resolution
- Frame skipping in the MAIN() loop — processes every Nth frame but still displays every frame, cutting inference calls without freezing the video
- Model loaded once in INIT(), outside the loop — avoids reloading a heavy model on every frame
- Confidence threshold filter in FILTER_NMS() — discards weak boxes early so fewer candidates reach drawing/NMS
- Non-Max Suppression (NMS) in FILTER_NMS() — removes duplicate overlapping boxes for the same object, correctness with multiple objects
- FOR EACH over a dynamic-length list in FILTER_NMS() and DRAW_BOX() — the same code path works for 0 objects or 50 with no change
- Non-blocking WAIT_KEY(1) in DISPLAY() — never stalls the loop waiting on keyboard input

7. Worked Trace — One Frame With 3 Detected Objects

A concrete walk-through of MAIN() on a single frame makes the loop behaviour verifiable and shows the pipeline scales to multiple objects without extra code paths.

Step	Variable / Action	Value after this step
1	frame = CAPTURE_FRAME(camera)	480x640 BGR image
2	blob = PREPROCESS(frame)	300x300 normalized blob
3	rawDetections = DETECT(model, blob)	5 raw candidates (incl. 2 low-confidence, 1 duplicate box)
4	FILTER_NMS() — confidence pass	3 candidates kept (conf >= 0.5): person 0.91, dog 0.77, chair 0.55
5	FILTER_NMS() — NMS pass	3 boxes remain (no overlap above 0.4 IoU) -> finalBoxes = 3
6	DRAW_BOX() FOR EACH (3 iterations)	3 rectangles + 3 labels drawn: "person 91%", "dog 77%", "chair 55%"
7	DISPLAY(frame)	Annotated frame shown; loop continues to next frame in < 1 frame's time

8. Summary

- Capture, detect, draw and display are split into 7 independent functions, called in a fixed order by MAIN().
- The WHILE loop in MAIN() delivers continuous real-time processing; BREAK gives a clean exit.
- A single FOR EACH loop over detections[] handles zero, one, or many objects with no extra logic.
- Frame skipping, a fixed input size, one-time model loading, and confidence + NMS filtering keep the pipeline within limited CPU/memory budgets while staying real time.